RLHFless: Serverless Computing for Efficient RLHF

Rui Wei
rwei7@stevens.edu
Stevens Institute of Technology

Hanfei Yu
hyu42@stevens.edu
Stevens Institute of Technology

Shubham Jain
sjain71@stevens.edu
Stevens Institute of Technology

Yogarajan Sivakumar
ysivakum@stevens.edu
Stevens Institute of Technology

Devesh Tiwari
d.tiwari@northeastern.edu
Northeastern University

Jian Li
jian.li.3@stonybrook.edu
Stony Brook University

Seung-Jong Park
seung-jong.park@mst.edu
Missouri University of Science & Technology

Hao Wang
hwang9@stevens.edu
Stevens Institute of Technology

arXiv:2602.22718v1 [cs.AI] 26 Feb 2026

Abstract

Reinforcement Learning from Human Feedback (RLHF) has been widely applied to Large Language Model (LLM) post-training to align model outputs with human preferences. Recent models, such as DeepSeek-R1, have also shown RLHF's potential to improve LLM reasoning on complex tasks. In RL, inference and training co-exist, creating dynamic resource demands throughout the workflow. Compared to traditional RL, RLHF further challenges training efficiency due to expanding model sizes and resource consumption. Several RLHF frameworks aim to balance flexible abstraction and efficient execution. However, they rely on serverful infrastructures, which struggle with fine-grained resource variability. As a result, during synchronous RLHF training, idle time between or within RL components often causes overhead and resource wastage.

To address these issues, we present `RLHFless`, the first scalable training framework for synchronous RLHF, built on serverless computing environments. `RLHFless` adapts to dynamic resource demands throughout the RLHF pipeline, pre-computes shared prefixes to avoid repeated computation, and uses a cost-aware actor scaling strategy that accounts for response length variation to find sweet spots with lower cost and higher speed. In addition, `RLHFless` assigns workloads efficiently to reduce intra-function imbalance and idle time. Experiments on both physical testbeds and a large-scale simulated cluster show that `RLHFless` achieves up to 1.35× speedup and 44.8% cost reduction compared to the state-of-the-art baseline.

1 Introduction

Motivation. Large Language Models (LLMs) have advanced rapidly and reshaped generative Artificial Intelligence (AI). The LLMs are first pre-trained on massive corpora with trillions of tokens from the internet to acquire broad knowledge [5]. They are then fine-tuned on high-quality, human-annotated instruction data to specialize for target tasks and domains. However, prior studies show that LLMs can still behave improperly and may generate harmful or unwanted content [10, 44, 81]. To address these issues, recent work applies reinforcement learning (RL) optimization to LLMs, aligning model behavior with human preferences through Reinforcement Learning from Human Feedback (RLHF). RLHF has been widely adopted in the latest LLMs [12, 43, 44, 60, 81] to enhance performance on challenging tasks such as mathematical problem solving [44, 68], answering scientific questions [23, 56], and code generation [12, 13]. RLHF training is significantly more resource-intensive and complex than traditional RL. While both follow a similar staged training paradigm, existing RL frameworks are not designed for the large-scale demands of RLHF. As model sizes grow and training is distributed across multiple GPUs and machines, the overhead from data transmission and model weight synchronization becomes a major bottleneck, limiting overall efficiency.

Limitations of state-of-art approaches. RL separates data sampling from model updates, but the two processes remain interdependent. Model updates must wait for sampling to finish in order to use the most recent training data, while sampling is delayed by model updates because it needs synchronized model weights. While traditional RL training already suffers from significant resource idleness on serverful frameworks [33, 37], RLHF amplifies this inefficiency due to the demands of running LLMs and heavy data transmission. Several works [14, 19, 30, 86] attempt to improve utilization by converting synchronous RLHF into asynchronous or off-policy execution. However, prior studies [14, 30, 41, 79] have shown that stale data introduced by asynchronous training can degrade convergence stability and final training quality. More recent synchronous RLHF systems [25, 56, 84] improve efficiency through careful model placement and parallelism strategies, but they rely on serverful infrastructures with static resource allocation, which inherently suffer from unavoidable resource inefficiency and cannot dynamically adapt to the execution characteristics of RLHF. Serverless RL frameworks [71, 73, 74] shed light on this problem by demonstrating that dynamically launching and releasing components on demand can substantially reduce idleness and training costs. However, existing serverless solutions for traditional RL cannot be directly applied to RLHF systems, and building a serverless RLHF framework is challenging due to fundamental differences between RLHF and traditional RL, such as the model scale and transmission overhead mentioned above. Moreover, the autoregressive nature of LLM generation and the continuously updated model weights lead to dynamic resource demands, both within a single step and across different training steps.

Key insights and contributions. Existing RLHF algorithms are highly sampling-heavy, where sequence generation dominates both execution time and resource consumption throughout training [54, 75, 77]. By analyzing existing synchronous RLHF workloads, we identify three key insights. First, tight dependencies between sampling and learning, combined with repeated key-value (KV) cache

computation during parallel generation, lead to substantial and unnecessary resource wastage. Second, long-tail responses cause stragglers during sequence generation and leave allocated resources idle. Third, workloads vary across training steps, as response lengths for the same prompt may increase or decrease over time, making static resource allocation inefficient. Motivated by these observations, we present `RLHFless`, the first scalable serverless framework for synchronous RLHF training, which improves efficiency and reduces cost. Before each training step, `RLHFless` constructs an optimized generation plan. It first analyzes the prompt dataset and training configuration to deduplicate KV cache computation under parallel generation. Next, `RLHFless` estimates the upcoming workload and dynamically scales the number of sampling actors[1] to balance training speed and resource cost. `RLHFless` further groups prompts with similar predicted response lengths to mitigate stragglers and reduce intra-actor imbalance. During execution, `RLHFless` places actors with locality awareness, prioritizing heavy workloads on nearby GPUs to better overlap communication and computation. Our main contributions are summarized as follows:

- We present `RLHFless`, the first serverless training framework for RLHF that reduces cost while accelerating training.
- We design a deduplicated prefill mechanism that eliminates redundant KV cache computation in RLHF.
- We propose a response-length prediction method with a cut-and-migrate backoff strategy to guide workload scheduling and actor scaling.
- We implement `RLHFless` on top of VERL [56] and evaluate it on both a physical cluster with two 8-GPU instances and a simulated large-scale cluster. Extensive experiments show that `RLHFless` reduces cost by up to 44.8% and improves training speed by up to 1.35× compared to the state-of-the-art baseline.

Experimental methodology and artifact availability. We implement `RLHFless` on top of VERL [56], a widely used open-source RLHF framework, and deploy it on a GPU cluster with two AWS EC2 `g6e.48xlarge` instances, each with 8 NVIDIA L40S GPUs, 192 AMD EPYC 7R13 vCPUs, and 384GB of system memory. Additionally, we use Vidur [1] to run large-scale simulations. We evaluate `RLHFless` using representative synchronous RLHF algorithms, including Proximal Policy Optimization (PPO) [81] and Group Relative Policy Optimization (GRPO) [54], on standard benchmarks covering mathematics (GSM8K [7]), scientific reasoning (GPQA [50]), and code generation (LiveCodeBench [28]). Following prior work [51, 74], we report per-step execution time and GPU×second cost to capture both performance and resource efficiency. We will release the source code and experimental traces upon acceptance.

Limitations of the proposed approach. `RLHFless` primarily focuses on synchronous RLHF training. While the actor scaling and prompt assignment designs in `RLHFless` are orthogonal and could be integrated into asynchronous RLHF systems [14, 30] with moderate adaptations, we leave such extensions to future work.

[1]An actor is a separate component that hosts a model replica for response generation.

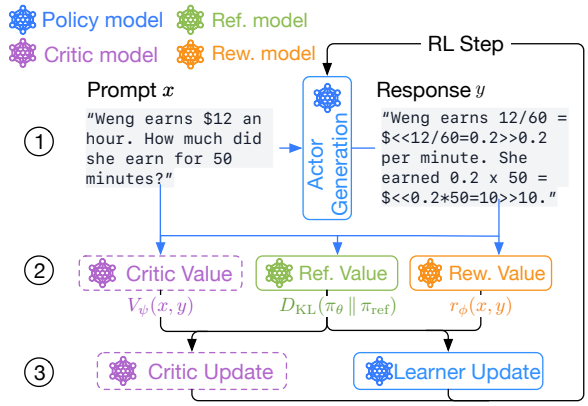

Figure 1: RLHF's dataflow in one training step, including the generation phase, preparation phase, and learning phase. Some details, like the use of the critic model, can vary depending on the specific algorithm used.

2 Background and Motivation

2.1 RLHF *vs.* General RL

Reinforcement Learning from Human Feedback (RLHF) is a method used to align the behavior of a language model with human preferences by incorporating feedback into the training loop. Rather than relying solely on supervised data, RLHF frames the alignment task as an RL problem, where a policy model (i.e., the LLM) π_θ is optimized to generate outputs that maximize the predicted reward provided by the pre-trained Reward Model (RM) $r_\phi(x, y)$. RLHF training contains multiple iterative training steps, and each training step involves three main phases, shown in Fig. 1:

Phase-1 Generation: For a batch of prompts x from the prompt dataset, multiple sampling ***actors*** will be launched using the policy π_θ to generate one or more candidate responses y. Each actor maintains its own replica of the policy model.

Phase-2 Preparation: The RM $r_\phi(x, y)$ assigns a scalar score to each generated response. To prevent the policy from deviating too far from its original behavior, many algorithms [31, 54, 81] incorporate a Kullback–Leibler (KL) divergence penalty between the current policy π_θ and the reference model π_{θ_0}.

Phase-3 Learning: A ***learner***, equipped with a training engine [48, 78], is responsible for computing gradients and updating the policy model using the data produced from previous phases.

Why existing RL frameworks cannot fit? Existing RL frameworks [33, 37, 63, 71, 73, 74, 85] are not designed to support the scale and complexity of RLHF for LLMs. Traditional RL workloads involve small models that can be trained on a small set of GPUs and rely on simple, synchronous architectures [33, 37, 71, 73, 85]. In contrast, LLM training requires distributing models across multiple GPUs using techniques like data, tensor, and pipeline parallelism [25, 48, 56, 78], along with specialized inference [29, 57, 80] and training [48, 78] engines. Moreover, scaling up the sampling actors in RLHF increases the need for synchronized replicas, which leads to expensive and slow weight synchronization [56]. In such a case, various RLHF-optimized frameworks [25, 55, 56, 65, 70] have been implemented to address those new challenges.

2.2 Limitations of Serverful RLHF Systems

RLHF faces several key challenges stemming from both its RL nature and the use of LLMs. The RL loop inherits the resource inefficiency commonly observed in traditional RL, caused by idle time between dependent components. For the LLM, the policy model generates responses with different lengths, and is continually updated, leading to varying workloads and dynamic resource demands. Existing RLHF frameworks [25, 56, 86] rely on serverful infrastructures and fail to address the issues both across the RL loop and within individual RLHF components.

1) From the outer RL loop: Idle components incur resource wastage on serverful platforms. In this setup, components from different RLHF phases have inter-dependencies, which often result in idle GPU resources during certain stages. Fig. 2(a) shows the workflow of GRPO [54], the default RLHF algorithm in DeepSeek R1 [12], running in standalone placement where each model uses a separate set of GPUs. Fig. 2(b) shows the latency breakdown in a training step for one of the GRPO model placement solutions provided by VERL [56], a commonly adopted RLHF framework, using Qwen2.5-3B model [4] with GSM8k dataset [7]. While some RLHF works [14, 19, 20, 86] attempt to address this issue by making the staged RLHF work asynchronous, prior research has shown that asynchronous training can degrade final training quality due to the inconsistencies introduced by stale data [14, 30, 41, 79]. Existing advanced synchronous RLHF solutions [25, 55, 56] address this by optimizing model placement strategies—selectively co-locating different components on the same set of resources. However, this presents a difficult trade-off: placing components separately leads to GPU under-utilization, while co-locating them can easily cause out-of-memory (OOM) errors, model context switching latency, or performance interference between components [55, 56, 83].

2) Across training steps: Dynamic response lengths make static resource allocation inefficient. Recent studies have shown that during RLHF's generation phase, the average response length for each training step keeps changing with an increasing or decreasing trend [12, 58, 60]. We run experiments with different RLHF algorithms and model combinations using the GSM8k dataset to verify this pattern, shown in Fig. 4(b). For most large-scale reasoning models, the response length will get longer as the training round increases [12, 60]. Longer response lengths prolong inference time due to LLMs' token-level auto-regressive generation, which indicates a dynamic resource demand for each training step. Serverful platforms, with typically static resource allocation and long startup times, are unable to adapt to such dynamic resource demands.

3) During each training step: Redundant KV cache calculation causes unnecessary costs. When an LLM generates a response, the prompt is first processed once to build the KV cache during prefill, then tokens are produced using this cache during decoding [11, 64]. Many RLHF algorithms generate multiple responses per prompt [54, 75, 77, 81], and many datasets share a large portion of prefixes among prompts. As a result, the same or similar prefill computation is often repeated unnecessarily. Prior LLM serving works reduce this cost by reusing the KV cache for prompts with shared prefixes [29, 35, 59, 67]. Some RLHF systems, such as VERL [56], apply this idea by splitting prompts into smaller batches, computing the KV cache once, and reusing it across batches within

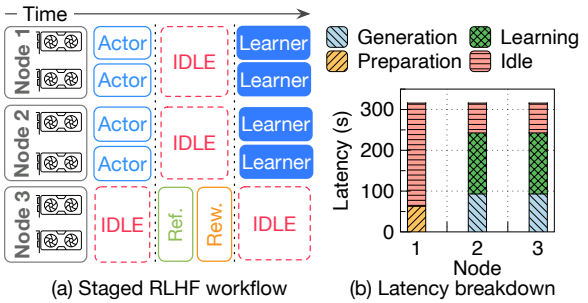

Figure 2: (a) RLHF's staged workflow causes idle time between components. (b) Idle components and (c) repeated calculation in RLHF lead to resource wastage.

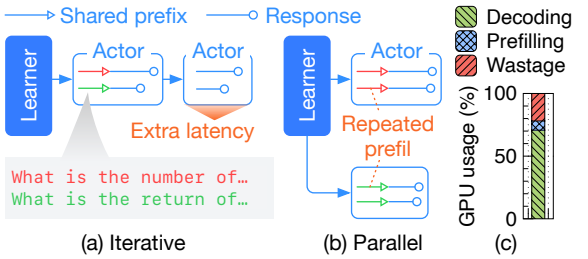

Figure 3: Existing generation strategies [25, 29, 56, 80] for sampling-heavy RLHF workloads, including (a) iterative generation, which introduces additional latency, and (b) parallel generation, which leads to (c) unnecessary KV recalculation.

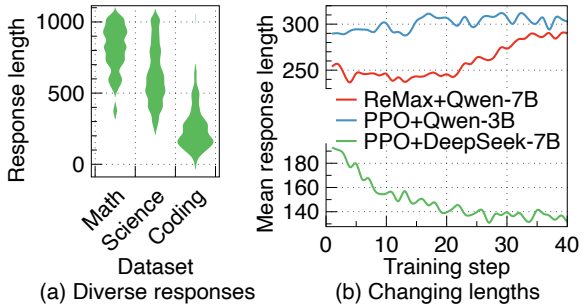

Figure 4: Dynamic resource demands caused by: (a) significantly varying response lengths across and within different types of datasets, such as mathematical problems [36], general scientific questions [50], and coding tasks [28]; (b) changing response lengths during RLHF training.

the same training step, as shown in Fig. 3(a). While this approach reduces duplicated prefill computation, it enforces sequential execution across batches. Under static GPU resource allocation, this serialization increases the overall generation time and limits parallelism. One may attempt to parallelize generation to improve throughput; however, this causes the same KV cache to be recomputed independently by different parallel workers, leading to unnecessary GPU resource waste, as illustrated in Fig. 3(b). Fig. 3(c) shows that approximately 22% of the total GPU-time is spent on redundant KV cache computation, when running GRPO with Qwen2.5-3B on GSM8K and generating four responses per prompt.

4) Inside the actors: Varying response lengths lead to GPU under-utilization. Sampling actors may consume significantly different amounts of compute resources due to the wide variation in response lengths. Within a single actor, some responses may terminate early after just a few decoding steps, while others may continue much longer, extending the generation phase even after most responses have completed [84]. This imbalance leads to GPU under-utilization, as certain compute resources remain idle while waiting for the longest responses to finish. Fig. 4(a) shows the imbalanced response lengths using the Qwen2.5-3B model, with 1024 maximal response length, in three different training datasets, including AIME2024 [36], GPQA [50], and LiveCodeBench [28].

2.3 Opportunities For Serverless RLHF

We propose the first serverless RLHF system by identifying the following opportunities to address the issues mentioned in §2.2.

Serverless can release idle components on time to mitigate wastage. Serverless elasticity and pay-as-you-go execution can lower training cost by launching components only when needed and promptly releasing them when idle, as shown in prior serverless RL systems [71, 73, 74]—motivating a serverless RLHF design.

Scaling actors to accommodate the changing resource demands. Serverless makes it feasible to adapt actor counts to shifting workloads and search for "sweet spots" where extra parallelism shortens step time enough to reduce total cost; achieving this requires a cost model to guide scaling decisions, and is harder on serverful platforms due to long startup latency [3, 17, 39].

Deduplicated prefill to mitigate repeated calculations in a serverless environment. By grouping prompts with shared prefixes ahead of time, and exploiting algorithms (*e.g.*, GRPO [54]) that sample multiple responses per prompt, we can compute prefill once and reuse it across responses—targeting redundant computation reduction (rather than serving throughput).

Grouping prompts with similar lengths in the same actor to address workload imbalance. Because actor cost depends on execution time, a few long responses can keep an actor alive while other resources sit idle; predicting response lengths and co-locating similar-length prompts lets some actors finish sooner and be released earlier.

3 Objectives and Challenges

We design a serverless RLHF training system, `RLHFless`, to achieve three primary objectives:

Speed up the end-to-end RLHF training process. Since the generation phase exhibits dynamic resource demands due to varying response lengths, we aim to accelerate the process by adjusting the number of actors to meet these changing resource needs.

Mitigate redundant computation at each training step. Shared prompt prefixes and multi-response generation in many RLHF algorithms [54, 75, 77] cause repeated KV cache computation during generation, leading to unnecessary cost. We aim to reduce this overhead using a deduplicated prefill.

Improve resource efficiency inside RL components. The response lengths assigned to a single actor can vary significantly, and outlier-long responses can prolong the actor's execution time

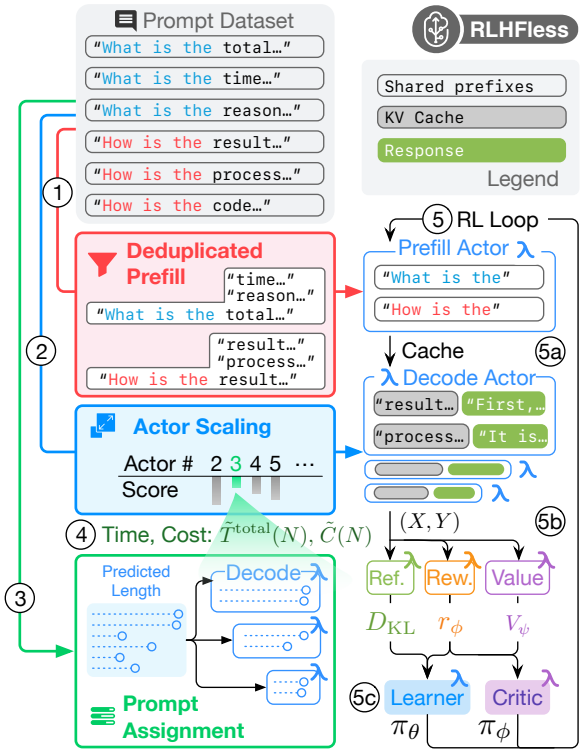

Figure 5: `RLHFless`'s overview.

while leading to low GPU utilization. Our goal is to reduce resource wastage caused by imbalanced workloads within each actor.

To achieve the goals, three challenges must be addressed:

How to trade off between speedup and cost? Scaling up the number of actors can speed up the generation phase, but usually increases resource cost. To find a balance, or even a sweet spot, with minimal cost and shortest time, it's necessary to estimate both cost and execution time accurately.

How to improve prefill efficiency in RLHF? Using a deduplicated prefill actor with fewer resources can reduce cost by computing each prompt only once. However, allocating too few resources may slow down the prefill stage, introducing latency into the overall RLHF training process.

How to minimize idle time within each actor? Grouping prompts with similar response lengths into the same decode actor can reduce GPU under-utilization and save cost. To do this effectively, we need an estimate of response lengths prior to assignment.

4 `RLHFless`'s Design

4.1 Overview

`RLHFless` is the first RLHF training system designed for serverless platforms. It leverages the elasticity and fine-grained resource management of serverless computing to facilitate RLHF training. Fig. 5 illustrates `RLHFless`'s architecture and workflow. `RLHFless` executes five main steps in every training step:

Step (1): Setting up the prefill actor. `RLHFless` analyzes the prompt dataset or algorithm-specific hyperparameters to identify a deduplicated batch of shared prefixes. These shared prompts are

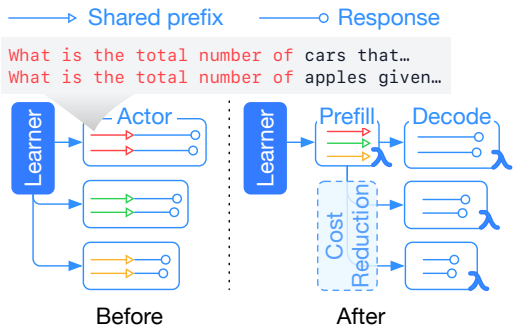

Figure 6: RLHFless's deduplicated prefill design.

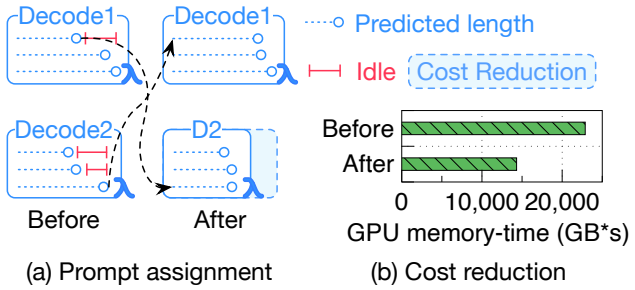

Figure 7: (a) RLHFless's prompt assignment design, and (b) the potential benefits that can be achieved under GRPO training with Qwen2.5-3B [4] model and GSM8k [7] dataset.

assigned to a dedicated prefill actor, which computes the initial KV cache to be reused by all decode actors during the decoding stage.

Step ②: Iterating all available actor numbers. RLHFless employs a cost-aware planning module to estimate both execution time and total cost across a range of possible actor counts, with the corresponding prompt assignment strategies.

Step ③: Prompt assignment among decode actors. When calculating the scaling score with a given actor number and workload, RLHFless first generates a corresponding prompt assignment plan for better cost and time estimation. RLHFless utilizes historical training data to predict the response length of each prompt in the current training batch. It then sorts prompts by estimated length and evenly distributes them into the N' decode actors, grouping prompts with similar lengths to minimize resource wastage and improve actor efficiency.

Step ④: Scaling the decode actors. The prompt assignment module returns the workload assignment plan given the actor number and its corresponding execution time and cost estimation. Based on those results, RLHFless's actor scaling module selects the optimal number of actors N' and its corresponding workload assignment plan by balancing speedup with resource efficiency.

Step ⑤: Executing RLHF loop. After generating the execution plan for the generation phase of the next training step, RLHFless initiates the step and executes the three RL phases, including (5a) generation, (5b) preparation, and (5c) learning, following the structure of conventional RLHF systems. The training process repeats the five steps until completion.

4.2 Deduplicated Prefill

RLHF algorithms [2, 24, 54, 75, 77] have duplicated KV cache computations in the generation phase of a training step due to two reasons: 1) shared prefixes among prompts in the training dataset, and 2) multiple responses generated for a single prompt. As a result, the number of unique KV cache computations required in the prefill stage is often less than the number of response generations performed in the decode stage. Existing serverful RLHF frameworks [25, 56] ignore this characteristic and allocate static resource sizes throughout training. In contrast, serverless computing offers a pay-as-you-go model, allowing us to reduce cost by applying a deduplicated prefill-decode design. This enables allocating only the necessary resources for the prefill actor and computing each unique KV cache exactly once. Since the prefill actor is compute-bound, similar to the learner [56, 83], we unify resource allocation and model parallelism strategy for both components.

We next estimate its maximum prefill batch size B_{prefill} using a profiling-based capacity model. RLHFless then chooses a target shared-prefix length L so that the prefill actor is filled to capacity with deduplicated prompts. Concretely, let X be the training prompt set and define a candidate prefix length $L \in [L_{\min}, L_{\max}]$, where

$$L_{\min} = \min_{x \in X} \|x\|, \quad L_{\max} = \max_{x \in X} \|x\|.$$

For a candidate prefix length L, let $D(L)$ be the number of unique prefixes of length L in X (*i.e.*, the count of deduplicated prompts at prefix length L). RLHFless sweeps $L \in [L_{\min}, L_{\max}]$ and selects the largest L such that

$$D(L) \leq B_{\text{prefill}}.$$

As L increases, $D(L)$ decreases; when $D(L)$ first satisfies $D(L) \leq B_{\text{prefill}}$, the prefill actor is fully utilized and computes the maximum amount of deduplicated KV cache within its capacity.

4.3 Prompt Assignment

Due to the varying lengths of responses generated within a single sampling actor, GPU utilization decreases when most sequences finish early and only a few long responses remain. Previous work [84] proposes a cut-and-migrate strategy to interrupt sequence generation in the middle, and migrate all unfinished responses to one actor to save resources for other tasks (*e.g.*, tasks in preparation phase). However, such a cut-and-migrate method introduces extra overhead due to the data transmission when migrating unfinished responses from one actor instance to another. Moreover, this method interrupts all actors simultaneously, harvesting resources only in a coarse-grained manner and conflicting with the fine-grained resource management principles of serverless computing.

To enable efficient cut-and-migrate in serverless environments, we propose a novel dynamic ranking-based prompt assignment strategy shown in Fig. 7(a), which significantly reduces the transmission cost by minimizing the number of cut responses. This approach releases idle resources at a finer granularity by allowing actors to terminate at different times, while keeping the workload balanced within each actor. As shown in Fig. 7(b), grouping prompts with similar predicted response lengths into the same actor can reduce this inefficiency by allowing early termination for each actor, thereby saving GPU time and cost. To enable this, we require a response length predictor to estimate workloads ahead of time.

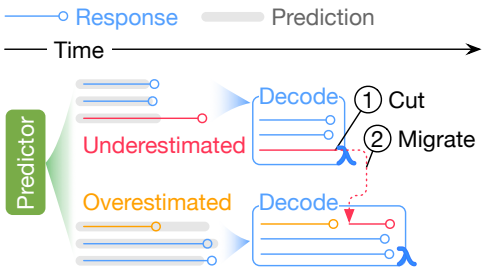

Figure 8: RLHFless's cut-and-migrate strategy. RLHFless cuts unfinished underestimated responses and migrates them to available decode actors with open slots, which have been released by completed, overestimated responses.

Response length estimation. Accurately estimating response generation workloads in advance is essential for assigning prompts to actors in a cost-efficient way. Specifically, we need to predict the expected response length for each prompt before launching the actors that will process it. Several recent works have proposed response length prediction methods to improve LLM serving efficiency [16, 46, 82]. However, these approaches typically require either finetuning the LLM to perform self-prediction [82] or training an external proxy model [16, 46]. Both options incur non-trivial training and prediction costs, adding complexity by introducing an extra training phase for the predictor. In addition, response length prediction remains inherently difficult due to the stochastic nature of LLM outputs. Even state-of-the-art (SOTA) methods that improve throughput for LLM serving still suffer from limited token length prediction accuracy [16, 46, 82].

A key advantage in the context of RLHF is that we have access to both the training dataset and historical response length data from earlier training steps. Unlike LLM serving, where predictions must be made from scratch for each user input, RLHF training involves repeated sampling over a fixed set of prompts. Moreover, our system design does not require exact response length estimates; rather, it only depends on the relative ranking of prompts by expected response length. As long as this ranking remains relatively stable across steps, we can make effective assignment decisions. To validate this, we track the response length changes for each prompt throughout the PPO training process on the GSM8K dataset, using the Qwen2.5-3B model with a maximum token limit of 2048. Most response lengths exhibit consistent trends over time and do not fluctuate drastically between steps. Approximately 70% of the response lengths vary by no more than 50 tokens across two consecutive epochs, and around 90% change by no more than 100 tokens per epoch. This suggests that we can use the historical response lengths observed in the previous training steps as a reliable estimator for the current step, eliminating the need for costly model-based predictors [16, 46, 82]. As for the first epoch without historical length data, we can make use of the ground truth answers from the training dataset as a temporary estimation. We adopt the common exponentially weighted moving average (EWMA) algorithm to take historical data as input and output the length estimation for each prompt.

Cut-and-migrate in RLHFless. Even with reasonably accurate response length estimation, mis-assignments can still occur. For example, if prompts with long actual responses are incorrectly assigned to actors intended for short responses, the resulting execution delay can reduce overall cost efficiency. To address this issue, we propose a novel cut-and-migrate strategy. Unlike existing methods that terminate all actors' executions simultaneously, RLHFless dynamically corrects prediction errors during execution, preserving the benefits of prompt assignment.

We classify the prediction errors into two categories: 1) Over-estimation: When a prompt with a short actual response is placed in an actor designed for long responses, GPU is under-utilized as the prompt finishes early and the remaining time is wasted. 2) Under-estimation: When a prompt with a long response is placed in an actor assigned to short responses, it prolongs the actor execution time. To mitigate the impact of prediction errors, we apply a cut-and-migrate strategy, inspired by existing work [84], shown in Fig. 8. During execution, if an actor is delayed due to long-running prompts and there are other available running actors, we terminate those generations and migrate them to the actors with available slots, which are released when over-estimated responses finish generation. This approach helps reduce tail latency and improves overall GPU utilization. The timing to terminate an actor is controlled by a cutting threshold $\tau \in (0, 1]$. An actor is terminated when $\tau \times 100\%$ of its assigned responses have completed generation. This threshold balances the trade-off between waiting for stragglers and minimizing resource wastage, allowing the system to release most of the GPU resources while offloading the remaining long responses to other actors.

4.4 Cost-Aware Actor Scaling

The average response length varies throughout RLHF training, causing dynamic execution times across training steps in traditional RLHF frameworks. With static resource allocation, this either results in resource wastage when fewer resources are needed or slow decoding when long responses cannot be processed efficiently. Thus, even if existing serverful RLHF systems can optimize generation strategy, they cannot consistently reach a sweet spot of low cost and fast speed, since workloads change at each step while resources and actor numbers remain fixed. This highlights the opportunity to dynamically adjust the number of actors to match changing resource demands.

Previous studies [71, 73] have demonstrated the potential of serverless computing's elasticity for actor scaling in traditional RL training, highlighting the possibility of implementing cost-aware actor scaling in a serverless RLHF system. However, scaling the number of actors in RLHF differs from traditional RL. Previous actor scaling methods either rely on a pretrained scheduler [73], which introduces additional costs, or utilize the Hessian matrix of the policy model to assess data needs [71]. The latter approach can incur significant computation overhead as the policy model size increases to billions of parameters. Furthermore, in typical RL, actor scaling alters the volume of training data by adjusting the number of actors, while in RLHF, changing the number of actors primarily adjusts the allocated GPU resources during the generation phase to suit the changing resource demands at each training step.

It has been well established in prior work that the generation phase is memory-bound, which can achieve higher performance

with increased data parallelism by deploying more model replicas to process prompts concurrently [32, 56, 83, 84]. In RLHF, each sampling actor maintains a copy of the policy model, meaning that the number of actors effectively determines the data parallelism size. As a result, increasing data parallelism directly corresponds to scaling up the number of actors. However, the speedup achieved by scaling up the number of actors may come at the cost of significantly increased resource consumption, which will further lead to exponential growth in training costs. This indicates the need for a cost-aware actor scaling design.

Balanced speedup with lower final costs. Estimating resource consumption before scaling up the number of actors is essential to avoid introducing unnecessary costs. The total cost for the N decode actors, denoted as C_{Decode}, can be estimated as

$$C_{\text{Decode}} := \rho \sum_{i=1}^{N} T(X_i, G_i) \times G_i,$$

where ρ is the unit cost of GPU usage, measured in dollars per GPU per unit time. X_i represents the set of prompts assigned to the ith decode actor, and G_i denotes the number of GPUs allocated to it. $T(X_i, G_i)$ is the estimated execution time for processing workload X_i using G_i GPUs. To estimate the execution time $T(X_i, G_i)$, we leverage profiled data collected using dummy batches prior to training. Specifically, we measure the time-per-output-token (TPOT) latencies in the decode step, under different maximum response lengths, following the approach used in prior work [6, 83]. For each prompt $x \in X_i$, the corresponding sequence length is estimated using the response length prediction module described in §4.3.

The end-to-end execution time for N decode actors, denoted as $T^{\text{total}}(N)$, is determined by the slowest (i.e., longest-running) actor and is computed as

$$T^{\text{total}}(N) := \max_{i \in [1,N]} T(X_i, G_i).$$

For all candidate actor counts $N \in [N_{\min}, N_{\max}]$, where $N_{\min}$ and $N_{\max}$ represent the predefined minimum and maximum number of decode actors, respectively, we normalize the execution time and cost to obtain

$$\tilde{T}^{\text{total}}(N) := \frac{T^{\text{total}}(N) - T^{\text{total}}_{\min}}{T^{\text{total}}_{\max} - T^{\text{total}}_{\min}}, \quad \tilde{C}(N) := \frac{C_{\text{Decode}}(N) - C_{\min}}{C_{\max} - C_{\min}},$$

where $T^{\text{total}}_{\min}$ and $T^{\text{total}}_{\max}$ are the minimum and maximum execution times observed over the range of N, and $C_{\min}$ and $C_{\max}$ are the corresponding cost bounds. To determine the optimal number of actors N', we formulate the objective as a weighted sum of the normalized time and cost:

$$N' := \underset{N \in [N_{\min}, N_{\max}]}{\arg\min} \left\{ \lambda \cdot \tilde{T}^{\text{total}}(N) + (1 - \lambda) \cdot \tilde{C}(N) \right\},$$

where $\lambda \in [0, 1]$ is a user-defined weight controlling the trade-off between execution time and cost. A larger λ prioritizes faster execution, while a smaller λ emphasizes lower cost.

4.5 Locality-Aware Actor Placement

The model synchronization overhead is one of the primary bottlenecks in RLHF training [56, 84]. This overhead occurs between the learning phase of the previous step and the generation phase of the current step. During synchronization, model weights are

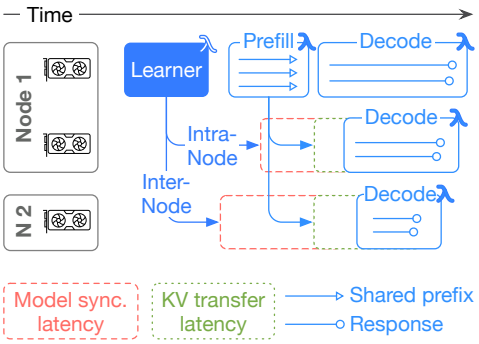

Figure 9: RLHFless's locality-aware actor placement.

transmitted from the learner to all sampling actors, introducing unavoidable transmission latency. Furthermore, applying RLHFless's actor scaling strategy to increase the number of actors, along with the disaggregated prefill design that separates actors into a prefill actor and multiple decode actors, can further amplify data transmission overhead due to increased communication. To address both model synchronization latency and KV transfer delays, RLHFless incorporates a locality-aware actor placement strategy, which combines the prompt assignment mechanism to properly place the actors, minimizing the end-to-end execution time in each training step.

As discussed in §4.3, RLHFless leverages prompt assignment to group prompts with similar expected workloads into the same decode actor. In this setting, the execution time of each actor is determined by the prompt with the longest response length within that group. Consequently, the end-to-end generation time for a training step is dictated by the actor with the longest generation workload. To optimize for this, we first co-locate the prefill actor with the learner on the same set of GPU resources. This reduces the transmission overhead for model weight synchronization between the learner and the prefill actor. Additionally, since both the learner and the prefill actor are compute-bound components that benefit from a larger model parallelism size [56, 83], co-locating them does not require changes to the existing parallelism strategy. Next, we co-locate the decode actor handling the longest response with the prefill actor, allowing it to start immediately without incurring latency for model weight and KV cache transfer. For additional decode actors introduced by actor scaling (§4.4), we prioritize those with heavier workloads for placement on GPU nodes that are physically closer to the learner and prefill actor (*e.g.*, within the same GPU node). Lighter workloads are assigned to more distant GPU resources. Since model weight synchronization can begin immediately after the learner finishes updating the model, while KV cache transfer can only start after the prefill stage completes, we can overlap transmission with execution. As illustrated in Fig. 9, transmission latency can be effectively hidden as long as the following condition holds for all $i \in \{2, \ldots, N\}$:

$$L^i_{\text{Model}} + L^i_{\text{KV}} + L^i_{\text{Decode}} \leq L_{\text{Prefill}} + L^1_{\text{Decode}},$$

where N is the total number of decode actors, L^i_{Model} and L^i_{KV} denote the model synchronization and KV cache transfer latency for the ith decode actor, respectively. L^i_{Decode} is the execution time of the ith decode actor, and L^1_{Decode} is the execution time of the decode

actor co-located with the learner and prefill actor. L_{Prefill} represents the execution time of the prefill actor. We add this constraint when scaling the actors to achieve a better actual performance, mitigating prolonged execution due to a larger-scale data transfer.

5 Implementation

We build `RLHFless` on top of an early version of VERL [56], one of the most commonly adopted RLHF frameworks, with integration of vLLM [29] as the inference engine and Fully Sharded Data Parallel (FSDP) [78] as the training engine.

Serverless cluster. We build a serverless cluster using Docker [38], a common approach in existing serverless environments. The Ray framework [40] manages compute resources for the serverless functions and coordinates communication among them.

Serverless functions. To mitigate the cold-start latency in the serverless RLHF system, we adopt an invocation prediction method, similar to prior work [45, 53]. Functions are prewarmed based on historical calls, using the shortest execution time from earlier steps plus a fixed buffer, ensuring they are ready before actual execution.

Model configuration. By default, `RLHFless` allocates the same number of GPUs to all decode actors. For internal settings (*e.g.*, tensor or pipeline parallelism), we follow VERL's documentation and tune hyperparameters to fit our GPUs.

Workload estimation. `RLHFless` 's core features require estimates of inference throughput and data transmission cost under specific parallelism and model sizes. We follow methods from prior work [1, 61, 83] and incorporate profiling results from dummy batches run during training initialization.

Data transmission. Redis [49] is used as an external distributed cache for data like historical response lengths, model checkpoints, and training traces. For model synchronization and KV cache transfer, we use NVIDIA Collective Communications Library (NCCL) [42] to avoid serialization overhead and enable direct communication.

Locality-aware function placement. To enable locality-aware placement, we profile GPU bandwidth and topology on the hardware testbeds. `RLHFless` uses these profiles with workload estimates to calculate cache transfer latencies. Model synchronization latency is relatively stable and can be estimated from historical results.

6 Evaluation

6.1 Experimental Setup

Workloads and datasets. To comprehensively demonstrate the performance benefits of `RLHFless`, we integrate multiple algorithms and datasets, and conduct experiments across various combinations of algorithms, datasets, and model sizes. Our algorithm choices include the widely adopted PPO [81], commonly used in early-stage RLHF, as well as GRPO [54], which replaces the critic model with repeated generations and forms the basis of many state-of-the-art RLHF algorithms [75, 77]. The training datasets span multiple domains, including the GSM8k mathematical reasoning dataset [7], the GPQA scientific question dataset [50], and the LiveCodeBench coding benchmark [28]. All models running on physical testbeds are based on the Qwen2.5 architecture [4], with tested sizes ranging from 3B to 7B. Due to budget constraints, we run large-scale simulations for larger Llama models [18, 62] using Vidur [1]. The maximal prompt length and response length are set to 1024 and 2048, respectively. We run each setting for three epochs and report the average per-step performance.

Hardware and system environment. We use two AWS EC2 `g6e.48xlarge` instances as our hardware testbed. Each instance has 8 NVIDIA L40S Tensor Core GPUs with 384GB VRAM in total, 192 AMD EPYC 7R13 vCPUs, and 384 GB CPU memory.

Software stack and configuration. The operating system of the testbeds is Ubuntu 22.04. The NCCL and CUDA versions are 2.27 and 12.8, respectively. We build `RLHFless` based on VERL 0.3.1.dev, and the selected inference engine is vLLM 0.8.3.

Baselines and comparisons. VERL [56] is a widely used open-source RLHF framework, and we build `RLHFless` on top of it. The core designs of `RLHFless` are general and can be integrated into other synchronous RLHF systems [25, 55, 86]. To ensure a fair comparison, we use VERL as our primary baseline. We additionally include RLHFuse [84] as a baseline to evaluate the effectiveness of prompt assignment in §6.4. For the length prediction comparison in §6.5, we train a model-based predictor following prior work [16, 46].

Methodology and experimental protocol. We record the execution time (in seconds) and the cost of each training step, and report the average per-step values over the entire training process. For each serverless function invocation, resource consumption is computed as the product of allocated resources and execution time. In practice, we measure GPU×second and use it as the cost proxy for each serverless function, following prior work [51, 74].

Reproducibility kit (AD/AE). The source code of `RLHFless` and the experimental traces will be publicly released if accepted.

`RLHFless`'s setting. Considering our resource limits and the model sizes used in experiments, each actor is allocated two GPUs for 3B training and four GPUs for 7B training. For the serverful baseline, we set the actor number to 3, as profiling shows this provides the best speed-cost balance under static resource allocation. More active actors for the entire training would significantly increase the cost. For the RLHF training hyperparameters in both `RLHFless` and the baseline, we follow the configurations provided by VERL [56], while tuning key parameters such as batch size and KV cache allocation rate to match our hardware testbed across different experimental setups. For the hyperparameters of `RLHFless`'s core components, we set the EWMA window size to 1, the cut-and-migrate threshold τ to 0.7, and the actor scaling weight λ to 0.7. These choices are justified and further analyzed in §6.5.

6.2 Overall Performance

We evaluate how `RLHFless` improves RLHF generation efficiency in terms of both speed and cost, by comparing it with a naive version of `RLHFless` built on VERL [56], where all core features are disabled. Fig. 10 shows that `RLHFless` speeds up RLHF training and lowers cost by (1) dynamically scaling actors to match changing resource demands, (2) eliminating repeated computation, and (3) releasing idle resources promptly. Training on GSM8k takes the longest, due to many long responses, which is consistent with the profiled results in Fig. 4(a). `RLHFless` delivers larger gains on GRPO than on PPO because GRPO is more sampling-heavy, leaving more room to reduce waste during generation. We also observe that as

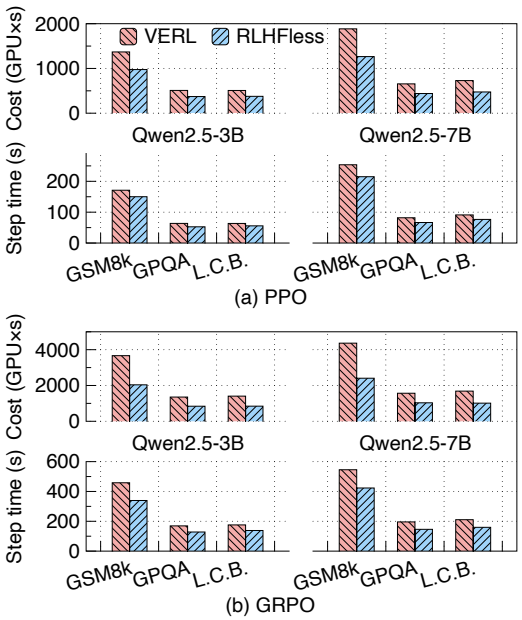

Figure 10: RLHFless's overall performance evaluation.

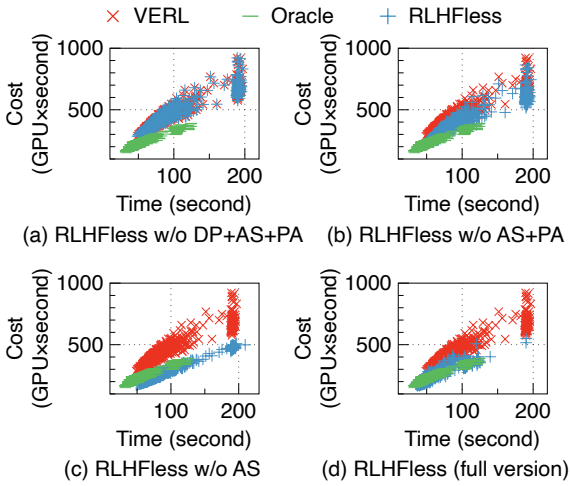

Figure 11: RLHFless's ablation study (DP: Deduplicated Prefill; AS: Actor Scaling; PA: Prompt Assignment). Each point shows the GPU cost and execution time of one training step. Points closer to the lower-left corner indicate lower cost and faster execution. Subfigure (a) shows RLHFless with all core features disabled. From (b) to (d), we enable one additional core feature at a time, illustrating the incremental benefits of each design.

training scale and workload grow, the benefits of RLHFless increase, suggesting its potential to adapt and deliver greater gains in super-large RLHF workloads. Overall, RLHFless speeds up training by up to 1.35× and reduces cost by up to 44.8%.

6.3 Ablation Study

We evaluate the effectiveness of RLHFless's core features—deduplicated prefill, prompt assignment, and actor scaling—via an ablation study using several system variants. We run experiments using the GRPO algorithm and the Qwen2.5-3B model on the GSM8k dataset. The study considers four variants of RLHFless: 1) RLHFless *w/o DP+AS+PA*: raw RLHFless based on VERL, with no deduplicated prefill, no prompt assignment, and no actor scaling; 2) RLHFless *w/o AS+PA*: RLHFless with only deduplicated prefill enabled, and without prompt assignment and no actor scaling; 3) RLHFless *w/o AS*: RLHFless with no actor scaling; and 4) RLHFless (full version): RLHFless with all three core features enabled. We further compare these variants against a VERL baseline and an oracle baseline, where the oracle simulates optimal scheduling with 100% accurate length estimates.

We visualize generation cost and execution time in Fig. 11. Each point represents one training step; more points closer to the lower-left indicate better performance with overall lower cost and shorter time. Fig. 11(b) shows the cost reduction from deduplicated prefill relative to the raw version in Fig. 11(a). While the reduction is modest in this setup, its magnitude is dataset- and algorithm-dependent and can be larger under different conditions. Fig. 11(c) demonstrates that prompt assignment with cut-and-migrate further reduces GPU cost by grouping prompts of similar length within the same actor. In rare cases, if cut-and-migrate is triggered late, migrations can introduce extra latency (*e.g.*, the rightmost outlier). Overall, transmission overhead is typically overlapped with computation and does not increase end-to-end latency. Fig. 11(d) illustrates the speedup from actor scaling, which dynamically adjusts the number of actors to match workload demands. When additional resources are allocated for heavier steps, the shorter execution time offsets the added resources, so the total cost grows slowly or remains comparable while achieving higher throughput.

6.4 Effectiveness of RLHFless

To better explain how each core feature of RLHFless works and where its benefits come from, we run controlled experiments that enable only one feature at a time. We use the same dataset and algorithm settings as in §6.3.

Effectiveness of prompt assignment. Fig. 12 shows the per-actor workloads and execution times of RLHFless compared with RLHFuse [84]. Note that in RLHF it is challenging to reproduce identical generated content due to hardware-level precision loss [21, 76]. Thus, we run RLHFless with only prompt assignment enabled using PPO on GSM8K, record the execution traces, and use them to simulate the raw version with all features disabled. We report results randomly selected from four consecutive training steps. In Fig. 12(a), RLHFuse incurs higher GPU time because it applies a global cut-and-migrate strategy without prompt ranking, leading to uneven prompt distribution across the three actors. By contrast, Fig. 12(b) shows that enabling prompt assignment groups prompts with similar expected lengths within the same actor, allowing actors 2 and 3 with shorter workloads to finish earlier and be released sooner, further reducing the costs. As a result, RLHFless's prompt assignment achieves 12.8% end-to-end cost reduction compared to RLHFuse's global cut-and-migrate without ranking. The response migration in RLHFless adds negligible overhead since only limited outlier prompts are misassigned to the wrong actor, which will be cut and migrated to unfinished neighbor actors with available slots.

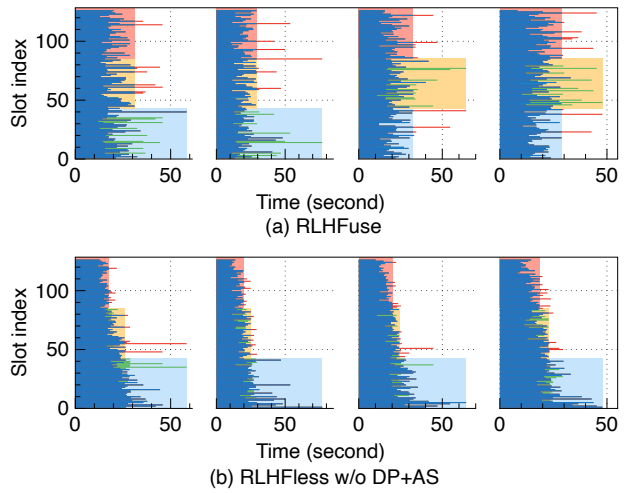

Figure 12: Effectiveness of RLHFless's prompt assignment. (DP: Deduplicated Prefill; AS: Actor Scaling; PA: Prompt Assignment.)

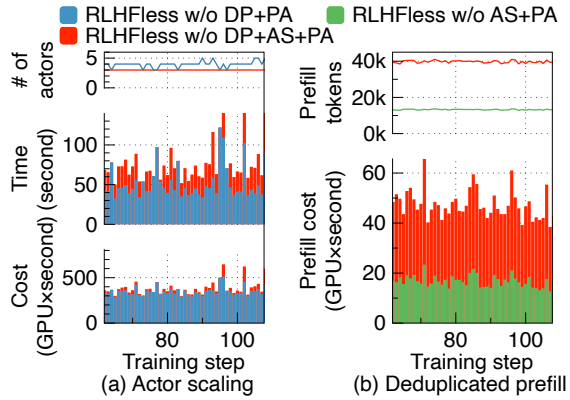

Figure 13: Effectiveness of RLHFless's (a) actor scaling and (b) deduplicated prefill. (DP: Deduplicated Prefill; AS: Actor Scaling; PA: Prompt Assignment.)

Effectiveness of actor scaling. We use the same setup described in §6.3 and compare the raw version of RLHFless with the same system with only actor scaling enabled for a fair comparison. Fig. 13(a) shows how the number of actors adapts when scaling is enabled. With scaling, RLHFless often finds a sweet spot where adding actors shortens the step time enough that the total cost, computed as # of GPUs × execution time after speeding up, is lower than using a fixed actor count. The results show that RLHFless can effectively estimate the next step's workload by referring to historical results, evaluate candidate actor counts by simulating their cost and time, and choose a setting that speeds up training without increasing cost in most cases.

Effectiveness of deduplicated prefill. To evaluate deduplicated prefill in RLHFless, we use the same setup as §6.3 and compare two variants: the raw RLHFless with no core features enabled and RLHFless with only deduplicated prefill enabled. We measure, for each training step, the total number of tokens processed and the total cost of the prefill stage during RLHF generation. Fig. 13(b) shows that RLHFless reduces the amount of prefill computation by eliminating repeated KV cache calculation. The magnitude of cost reduction depends on the RLHF configuration. For example, in Fig. 13(b), we run GRPO with three responses per prompt; computing the prefill once per prompt theoretically yields a 66% reduction in prefill cost.

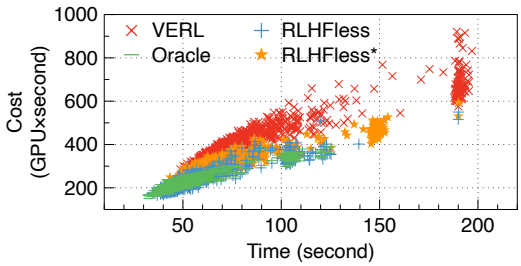

Figure 14: Sensitivity analysis of RLHFless's response length prediction.

6.5 Sensitivity Analysis

To test robustness, justify our choices, and show how prediction accuracy and hyperparameter settings affect performance, we repeat the setup mentioned in §6.3 and run experiments to analyze the sensitivity of each hyperparameter. We conduct the average per-step cost and time for each setting, shown in Figs. 14 and 15.

Sensitivity to response length prediction accuracy. In RLHFless, both actor scaling and prompt assignment depend on response length estimates. To test robustness against imperfect estimates, we replace our historical, training-step-based estimator with lower-accuracy predictors from prior work [16, 46], and evaluate the resulting performance. We integrate a model-based predictor into RLHFless, denoted as RLHFless*. Following prior works in LLM serving [16, 46], this predictor is attached to the policy model under training. It must be pre-trained on the target RLHF dataset, which takes approximately 11 GPU-hours, and achieves only about 44% accuracy. In principle, higher prediction accuracy should yield better decisions for actor scaling and prompt assignment. Fig. 14 shows that RLHFless* still achieves 1.22× speedup and reduces RLHF generation cost by 28.9% over the VERL baseline, despite its lower accuracy. This indicates that the predictor in RLHFless is modular and replaceable, and that RLHFless tolerates inaccurate estimates. The robustness comes from three factors: (1) RLHFless's prompt assignment relies on the ranking of expected lengths rather than exact values, (2) the cut-and-migrate backoff corrects harmful misassignments at runtime, and (3) actor scaling uses profiled TPOT latency curves, which dampens the effect of individual prediction errors.

Sensitivity to EWMA window size for length prediction. The window size is the number of past epochs to predict next-epoch lengths. We run fifteen epochs and vary the window from 1 to 4. Fig. 15(a) shows only marginal gains from larger windows. With few training epochs and often monotonic length trends, older data becomes stale and does not improve accuracy. We therefore set the window to 1 (use the latest epoch only) for simplicity.

Sensitivity to cut-and-migrate threshold τ. The threshold τ controls when to trigger the cut-and-migrate process, which terminates unfinished responses in one actor and moves them to other

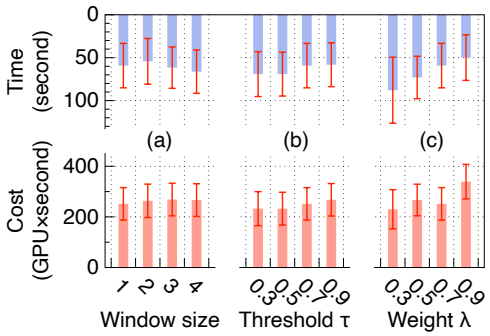

Figure 15: RLHFless hyperparameter sensitivity: (a) EWMA window size for length prediction, (b) cut-and-migrate threshold τ, and (c) actor scaling weight λ.

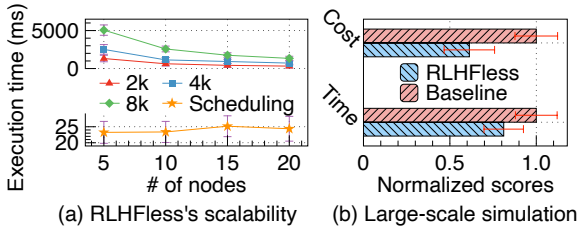

Figure 16: RLHFless scalability analysis. (a) Execution time for response generation with lengths from 2k to 8k tokens, along with scheduling overhead, under different cluster sizes. (b) Average normalized performance improvement from large-scale simulation experiments.

available actors. We run three epochs and vary $\tau \in [0.3, 0.9]$. As shown in Fig. 15(b), smaller τ triggers earlier cuts, which reduces cost by eliminating idle time in a finer granularity, but increases per-step time due to more migrations and state transfers. Additionally, costs do not decrease further when $\tau \leq 0.5$. Even if cut-and-migrate is triggered earlier, it can proceed only when other actors have available slots, so early cuts often end up waiting and yield limited savings. Thus, we choose $\tau = 0.7$ to balance time and cost.

Sensitivity to actor-scaling weight λ. The actor-scaling strategy uses a weight λ to balance cost versus speed. We run three epochs and vary $\lambda \in [0.3, 0.9]$. A higher λ prioritizes speed; a lower λ prioritizes cost. Although $\lambda = 0.5$ may seem like the most balanced setting, Fig. 15(c) shows that $\lambda = 0.7$ yields the best trade-off in our experiments, achieving the second-lowest cost and second-shortest step time. This is because the min-max normalization used in the objective does not fully preserve the original distribution of cost and time. Even after scaling to $[0, 1]$, their influence on the final score may still be uneven. While these hyperparameters may not generalize across all workloads, the observed trends are consistent and provide a practical starting point for tuning.

6.6 Scalability Analysis

To evaluate RLHFless at a larger scale under a limited budget, we implement a simulator for PPO generation on GSM8K using Vidur [1], a widely used LLM serving simulator, together with RLHF training traces from VERL [56]. We simulate a cluster with up to 20 nodes, each equipped with 8 NVIDIA H100 GPUs connected via NVLink. **RLHFless's scalability.** We use the Llama2-70B model with a generation batch size of 512 and scale the number of nodes, allocating two GPUs per actor. Fig. 16(a) shows that per-step generation time decreases as more resources are added under a fixed workload, indicating strong scaling. Meanwhile, the scheduling overhead of RLHFless remains stable and within 30 ms across all configurations.

Large-scale simulation performance. We then calculate the average returns of RLHFless across the simulation settings used previously, and report the normalized results in Fig. 16(b). As a result, RLHFless achieves 1.23× speedup and 38.7% cost reduction compared to the simulated RLHF generation baseline.

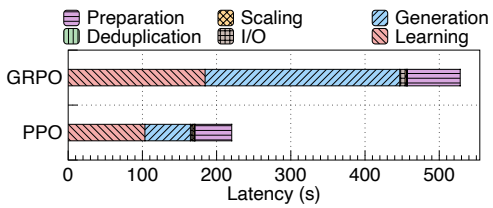

Figure 17: RLHFless's latency breakdown in a training step, including the preparation, generation, and learning phases, as well as deduplication latency, scaling latency (response-length prediction and cost/time estimation), and I/O latency.

6.7 Latency Breakdown

We evaluate the latency breakdown and overhead of RLHFless's core components within a single RLHF training step. Fig. 17 presents the breakdown for two representative RLHF algorithms: PPO and GRPO. The overhead introduced by RLHFless's core features, including prompt deduplication, response length prediction, and cost/time estimation, is negligible relative to the total step time. This is because all core components in RLHFless are model-free and lightweight, adding minimal computational burden to the training pipeline.

7 Related Work

LLM alignment methods. After Supervised Fine-tuning (SFT), LLM alignment methods can be broadly divided into RL-based and non-RL approaches [66]. Common RL-based methods include PPO [52, 81], GRPO [54], ReMax [31], REINFORCE [2], and REINFORCE++ [24]. Non-RL methods such as Direct Preference Optimization (DPO) [47] remove the RL loop by reformulating preference learning as supervised optimization, but have been shown to underperform RL-based methods on complex reasoning and coding tasks [69]. In this paper, we identify unique patterns of RL-based methods and design corresponding optimizations to improve the resource efficiency and accelerate the sampling process.

RL and LLM alignment frameworks. Traditional RL frameworks [22, 26, 33, 85] target small-scale models and do not scale to LLM alignment due to model size and communication overhead. Recent LLM alignment frameworks [8, 14, 19, 25, 30, 55, 56, 70, 84, 86] typically combine a high-throughput inference engine [9, 29, 80] with a distributed training backend [48, 57, 78]. To mitigate idle time in staged RLHF workflows, some systems adopt asynchronous and off-policy execution [14, 19, 30, 86], although prior work

shows that stale data can degrade training quality [41, 79]. Meanwhile, some systems optimize synchronous training through model placement and parallelism [25, 55, 56, 70]. All of these frameworks rely on serverful infrastructure and cannot adapt resource allocation at fine granularity across training steps. RLHFuse (FlexFusion) [84] offloads long-tail responses to dedicated GPUs, but its global cut-and-migrate strategy may incur extra overhead without workload-aware prompt assignment, as discussed in §6.4. In contrast, `RLHFless` introduces serverless execution with deduplicated prefill, actor scaling, and prompt assignment, which are orthogonal to existing synchronous frameworks.

Serverless computing for LLM. Serverless computing has gained increasing attention in LLM applications due to its flexibility, automatic scaling, and pay-as-you-go pricing model, especially in the context of LLM serving [15, 27, 34, 72]. ServerlessLLM [15] supports low-latency LLM serving by leveraging the large memory and storage capacities available on modern GPU servers. ENOVA [27] introduces a configuration recommendation module for automatic deployment across various GPU clusters, along with a performance monitoring module that enables auto-scaling. However, all of these serverless systems are focused solely on LLM serving and they are not designed for RLHF workloads.

8 Conclusion

This paper presents `RLHFless`, the first serverless framework for RLHF training. By targeting key inefficiencies in serverful RLHF systems—including idle time across dependent training phases, dynamic response lengths, and redundant KV cache computation—`RLHFless` leverages serverless execution for fine-grained resource adaptation. Through deduplicated prefill, prompt assignment, and cost-aware actor scaling, `RLHFless` reduces resource wastage both across the RLHF loop and within individual components. Experiments show that `RLHFless` achieves up to 1.35× speedup with 44.8% cost reduction on physical clusters, and an average 1.23× speedup with 38.7% cost reduction on a simulated large-scale GPU cluster, demonstrating the effectiveness of serverless RLHF training.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework For LLM Inference. arXiv:2405.05465 [cs.LG] https://arxiv.org/abs/2405.05465

[2] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740 [cs.LG] https://arxiv.org/abs/2402.14740

[3] Amazon Web Services. 2025. AWS Elastic Compute Cloud. https://aws.amazon.com/ec2/.

[4] Qwen and:, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] https://arxiv.org/abs/2412.15115

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[6] Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffenetti, Valerie Taylor, Murali Emani, and Venkatram Vishwanath. 2024. LLM-Inference-Bench: Inference Benchmarking of Large Language Models on AI Accelerators. arXiv:2411.00136 [cs.LG] https://arxiv.org/abs/2411.00136

[7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).

[8] Collosal-AI Corporation. 2023. Collosal-Chat: A Project to Implement LLM with RLHF. https://github.com/hpcaitech/ColossalAI/tree/main/applications/ColossalChat.

[9] NVIDIA Corporation. 2023. TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. https://github.com/NVIDIA/TensorRT-LLM.

[10] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2023. Safe RLHF: Safe Reinforcement Learning from Human Feedback. arXiv:2310.12773 [cs.AI] https://arxiv.org/abs/2310.12773

[11] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. arXiv:1901.02860 [cs.LG] https://arxiv.org/abs/1901.02860

[12] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv.org/abs/2501.12948

[13] Sujan Dutta, Sayantan Mahinder, Raviteja Anantha, and Bortik Bandyopadhyay. 2024. Applying RLAIF for Code Generation with API-usage in Lightweight LLMs. arXiv:2406.20060 [cs.CL] https://arxiv.org/abs/2406.20060

[14] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. arXiv:2505.24298 [cs.LG] https://arxiv.org/abs/2505.24298

[15] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 135–153. https://www.usenix.org/conference/osdi24/presentation/fu

[16] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. arXiv:2408.15792 [cs.LG] https://arxiv.org/abs/2408.15792

[17] Google Cloud Platform. 2025. GCP Compute Engine Instance. https://cloud.google.com/compute/docs/instances.

[18] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste

Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[19] Tyler Griggs, Sumanth Hegde, Eric Tang, Shu Liu, Shiyi Cao, Dacheng Li, Charlie Ruan, Philipp Moritz, Kourosh Hakhamaneshi, Richard Liaw, Akshay Malik, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Evolving SkyRL into a Highly-Modular RL Framework. Notion Blog.

[20] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, Huazhong Ji, Wenjie Liu, Yu Huang, Yixiang Zhang, Chenyi Pan, Jing Wang, Xin Huang, Chunsheng Li, and Jianping Wu. 2025. AsyncFlow: An Asynchronous Streaming RL Framework for Efficient LLM Post-Training. arXiv:2507.01663 [cs.LG] https://arxiv.org/abs/2507.01663

[21] Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. *Thinking Machines Lab: Connectionism* (2025). doi:10.64434/tml.20250910 https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[22] Matthew W. Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Nikola Momchev, Danila Sinopalnikov, Piotr Stańczyk, Sabela Ramos, Anton Raichuk, Damien Vincent, Léonard Hussenot, Robert Dadashi, Gabriel Dulac-Arnold, Manu Orsini, Alexis Jacq, Johan Ferret, Nino Vieillard, Seyed Kamyar Seyed Ghasemipour, Sertan Girgin, Olivier Pietquin, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, Sarah Henderson, Abe Friesen, Ruba Haroun, Alex Novikov, Sergio Gómez Colmenarejo, Serkan Cabi, Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Andrew Cowie, Ziyu Wang, Bilal Piot, and Nando de Freitas. 2020. Acme: A Research Framework for Distributed Reinforcement Learning. *arXiv preprint arXiv:2006.00979* (2020). https://arxiv.org/abs/2006.00979

[23] Zhenyu Hou, Pengfan Du, Yilin Niu, Zhengxiao Du, Aohan Zeng, Xiao Liu, Minlie Huang, Hongning Wang, Jie Tang, and Yuxiao Dong. 2024. Does RLHF Scale? Exploring the Impacts From Data, Model, and Method. arXiv:2412.06000 [cs.CL]

[24] Jian Hu, Jason Klein Liu, and Shen Wei. 2025. REINFORCE++: An Efficient RLHF Algorithm with Robustness to Both Prompt and Reward Models. arXiv:2501.03262 [cs.CL] https://arxiv.org/abs/2501.03262

[25] Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. 2024. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143 [cs.AI] https://arxiv.org/abs/2405.11143

[26] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G.M. Araújo. 2022. CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms. *Journal of Machine Learning Research* 23, 274 (2022), 1–18. http://jmlr.org/papers/v23/21-1342.html

[27] Tao Huang, Pengfei Chen, Kyoka Gong, Jocky Hawk, Zachary Bright, Wenxin Xie, Kecheng Huang, and Zhi Ji. 2024. ENOVA: Autoscaling towards Cost-effective and Stable Serverless LLM Serving. In *arXiv preprint arXiv:*.

[28] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv preprint arXiv:2403.07974* (2024).

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles.*

[30] Xiaocan Li, Shiliang Wu, and Zheng Shen. 2026. A-3PO: Accelerating Asynchronous LLM Training with Staleness-aware Proximal Policy Approximation. arXiv:2512.06547 [cs.LG] https://arxiv.org/abs/2512.06547

[31] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. 2024. ReMax: A Simple, Effective, and Efficient Reinforcement Learning Method for Aligning Large Language Models. arXiv:2310.10505 [cs.LG] https://arxiv.org/abs/2310.10505

[32] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. arXiv:2302.11665 [cs.LG] https://arxiv.org/abs/2302.11665

[33] Eric Liang, Richard Liaw, Philipp Moritz, Robert Nishihara, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. 2018. RLlib: Abstractions for Distributed Reinforcement Learning. arXiv:1712.09381 [cs.AI] https://arxiv.org/abs/1712.09381

[34] Mengfan Liu, Wei Wang, and Chuan Wu. 2025. Optimizing Distributed Deployment of Mixture-of-Experts Model Inference in Serverless Computing. arXiv:2501.05313 [cs.DC] https://arxiv.org/abs/2501.05313

[35] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. arXiv:2310.07240 [cs.NI] https://arxiv.org/abs/2310.07240

[36] Maxwell Jia. 2025. American Invitational Mathematics Examination (AIME) 2024 Dataset. https://huggingface.co/datasets/Maxwell-Jia/AIME_2024.

[37] Zhiyu Mei, Wei Fu, Jiaxuan Gao, Guangju Wang, Huanchen Zhang, and Yi Wu. 2024. SRL: Scaling Distributed Reinforcement Learning to Over Ten Thousand Cores. arXiv:2306.16688 [cs.DC] https://arxiv.org/abs/2306.16688

[38] Dirk Merkel et al. 2014. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal* (2014).

[39] Microsoft Azure. 2025. Microsoft Azure Virtual Machines. https://azure.microsoft.com/pricing/details/virtual-machines/.

[40] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. arXiv:1712.05889 [cs.DC] https://arxiv.org/abs/1712.05889

[41] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. 2025. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models. arXiv:2410.18252 [cs.LG] https://arxiv.org/abs/2410.18252

[42] NVIDIA. 2025. NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl.

[43] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[44] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL] https://arxiv.org/abs/2203.02155

[45] Olga Poppe, Qun Guo, Willis Lang, Pankaj Arora, Morgan Oslake, Shize Xu, and Ajay Kalhan. 2022. Moneyball: proactive auto-scaling in Microsoft Azure SQL database serverless. *Proc. VLDB Endow.* 15, 6 (Feb. 2022), 1279–1287. doi:10.14778/3514061.3514073

[46] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T. Kalbarczyk, Tamer Başar, and Ravishankar K. Iyer. 2024. Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction. arXiv:2404.08509 [cs.DC] https://arxiv.org/abs/2404.08509

[47] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.LG] https://arxiv.org/abs/2305.18290

[48] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) *(KDD '20)*. Association for Computing Machinery, New York, NY, USA, 3505–3506. doi:10.1145/3394486.3406703

[49] Redis. 2025. Redis Official Website. http://redis.io/.

[50] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. In *First Conference on Language Modeling*. https://openreview.net/forum?id=Ti67584b98

[51] Francisco Romero, Mark Zhao, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. Llama: A Heterogeneous & Serverless Framework for Auto-Tuning Video Analytics Pipelines. arXiv:2102.01887 [cs.DC] https://arxiv.org/abs/2102.01887

[52] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] https://arxiv.org/abs/1707.06347

[53] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX annual technical conference (USENIX ATC)*.

[54] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] https://arxiv.org/abs/2402.03300

[55] Gerald Shen, Zhilin Wang, Olivier Delalleau, Jiaqi Zeng, Yi Dong, Daniel Egert, Shengyang Sun, Jimmy Zhang, Sahil Jain, Ali Taghibakhshi, Markel Sanz Ausin, Ashwath Aithal, and Oleksii Kuchaiev. 2024. NeMo-Aligner: Scalable Toolkit for Efficient Model Alignment. arXiv:2405.01481 [cs.CL] https://arxiv.org/abs/2405.01481

[56] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 1279–1297. doi:10.1145/3689031.3696075

[57] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL] https://arxiv.org/abs/1909.08053

[58] Prasann Singhal, Tanya Goyal, Jiacheng Xu, and Greg Durrett. 2024. A Long Way to Go: Investigating Length Correlations in RLHF. arXiv:2310.03716 [cs.CL] https://arxiv.org/abs/2310.03716

[59] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. 2024. You Only Cache Once: Decoder-Decoder Architectures for Language Models. arXiv:2405.05254 [cs.CL] https://arxiv.org/abs/2405.05254

[60] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, and Zonghan Yang. 2025. Kimi k1.5: Scaling Reinforcement Learning with LLMs. arXiv:2501.12599 [cs.AI] https://arxiv.org/abs/2501.12599

[61] Wei Ming Thor. 2025. How To Calculate GPU VRAM Requirements for an Large-Language Model. ApX Machine Learning. https://apxml.com/posts/how-to-calculate-vram-requirements-for-an-llm Last updated: July 15, 2025.

[62] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] https://arxiv.org/abs/2307.09288

[63] Nicholas Ustaran-Anderegg, Michael Pratt, and Jaime Sabal-Bermudez. [n. d.]. *AgileRL*. https://github.com/AgileRL/AgileRL

[64] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] https://arxiv.org/abs/1706.03762

[65] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. TRL: Transformer Reinforcement Learning. https://github.com/huggingface/trl.

[66] Zhichao Wang, Bin Bi, Shiva Kumar Pentyala, Kiran Ramnath, Sougata Chaudhuri, Shubham Mehrotra, Zixu, Zhu, Xiang-Bo Mao, Sitaram Asur, Na, and Cheng. 2024. A Comprehensive Survey of LLM Alignment Techniques: RLHF, RLAIF, PPO, DPO and More. arXiv:2407.16216 [cs.CL] https://arxiv.org/abs/2407.16216

[67] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453 [cs.CL] https://arxiv.org/abs/2309.17453

[68] Wei Xiong, Hanze Dong, Chenlu Ye, Ziqi Wang, Han Zhong, Heng Ji, Nan Jiang, and Tong Zhang. 2024. Iterative Preference Learning from Human Feedback: Bridging Theory and Practice for RLHF under KL-Constraint. arXiv:2312.11456 [cs.LG] https://arxiv.org/abs/2312.11456

[69] Shusheng Xu, Wei Fu, Jiaxuan Gao, Wenjie Ye, Weilin Liu, Zhiyu Mei, Guangju Wang, Chao Yu, and Yi Wu. 2024. Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study. arXiv:2404.10719 [cs.CL] https://arxiv.org/abs/2404.10719

[70] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song, and Yuxiong He. 2023. DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-like Models at All Scales. arXiv:2308.01320 [cs.LG] https://arxiv.org/abs/2308.01320

[71] Hanfei Yu, Jacob Carter, Hao Wang, Devesh Tiwari, Jian Li, and Seung-Jong Park. 2025. Nitro: Boosting Distributed Reinforcement Learning with Serverless Computing. In *51st International Conference on Very Large Data Bases (VLDB)*.

[72] Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. 2025. fMoE: Fine-Grained Expert Offloading for Large Mixture-of-Experts Serving. arXiv:2502.05370 [cs.LG] https://arxiv.org/abs/2502.05370

[73] Hanfei Yu, Jian Li, Yang Hua, Xu Yuan, and Hao Wang. 2024. Cheaper and Faster: Distributed Deep Reinforcement Learning with Serverless Computing. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[74] Hanfei Yu, Hao Wang, Devesh Tiwari, Jian Li, and Seung-Jong Park. 2024. Stellaris: Staleness-Aware Distributed Reinforcement Learning with Serverless Computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC)*.

[75] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476 [cs.LG] https://arxiv.org/abs/2503.14476

[76] Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. 2025. Give Me FP32 or Give Me Death? Challenges and Solutions for Reproducible Reasoning. arXiv:2506.09501 [cs.CL] https://arxiv.org/abs/2506.09501

[77] Yu Yue, Yufeng Yuan, Qiying Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, TianTian Fan, Zhengyin Du, Xiangpeng Wei, Xiangyu Yu, Gaohong Liu, Juncai Liu, Lingjun Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Ru Zhang, Xin Liu, Mingxuan Wang, Yonghui Wu, and Lin Yan. 2025. VAPO: Efficient and Reliable Reinforcement Learning for Advanced Reasoning Tasks. arXiv:2504.05118 [cs.AI] https://arxiv.org/abs/2504.05118

[78] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. arXiv:2304.11277 [cs.DC] https://arxiv.org/abs/2304.11277

[79] Haizhong Zheng, Jiawei Zhao, and Beidi Chen. 2025. Prosperity before Collapse: How Far Can Off-Policy RL Reach with Stale Data on LLMs? arXiv:2510.01161 [cs.LG] https://arxiv.org/abs/2510.01161

[80] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104

[81] Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, Limao Xiong, Lu Chen, Zhiheng Xi, Nuo Xu, Wenbin Lai, Minghao Zhu, Cheng Chang, Zhangyue Yin, Rongxiang Weng, Wensen Cheng, Haoran Huang, Tianxiang Sun, Hang Yan, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. 2023. Secrets of RLHF in Large Language Models Part I: PPO. arXiv:2307.04964 [cs.CL] https://arxiv.org/abs/2307.04964

[82] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2023. Response Length Perception and Sequence Scheduling: An LLM-Empowered LLM Inference Pipeline. arXiv:2305.13144 [cs.CL] https://arxiv.org/abs/2305.13144

[83] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670 [cs.DC] https://arxiv.org/abs/2401.09670

[84] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, and Xin Jin. 2024. Optimizing RLHF Training for Large Language Models with Stage Fusion. *arXiv preprint arXiv: 2409.13221* (2024).

[85] Huanzhou Zhu, Bo Zhao, Gang Chen, Weifeng Chen, Yijie Chen, Liang Shi, Yaodong Yang, Peter Pietzuch, and Lei Chen. 2023. MSRL: Distributed Reinforcement Learning with Dataflow Fragments. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 977–993. https://www.usenix.org/conference/atc23/presentation/zhu-huanzhou

[86] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. 2025. slime: An LLM post-training framework for RL Scaling. https://github.com/THUDM/slime. GitHub repository. Corresponding author: Xin Lv.